

Accelerating a Regional Hyper-Resolution SWAT+ Model Without Changing the Science: a Reproducible Profiling-to-Parallelization Methodology and a Routing-Aware Shared-Memory Wavefront

Vahid Rafiei

Independent Researcher

`vahidr32@gmail.com`

ORCID: 0009-0009-8309-1895

Abstract

Process-based watershed models such as SWAT+ are increasingly applied at regional, hyper-resolution scale, where a single simulation becomes the bottleneck: one simulated year of a 57,998-hydrologic-response-unit (HRU) model takes about three minutes on one core, and calibration multiplies that by thousands of evaluations. Existing parallel SWAT efforts exploit ensemble parallelism (many independent runs) or grid-cell decomposition; the cost of a single simulation is left untouched. We present a reproducible methodology for accelerating the SWAT+ engine itself on shared-memory hardware without altering model results: profile, classify the bottleneck, fix by class, validate, re-measure. Profiling shows the dominant costs at scale are overhead, not hydrology—per-object array re-zeroing, derived-type copies, $\mathcal{O}(N^2)$ name-matching, and a serial routing critical path. We remove these and add an OpenMP parallelization that respects the routing directed acyclic graph: a topological level wavefront over the engine’s

command order, made reentrant by privatizing current-object state. The result separates two products: serial overhead removal accelerates the engine everyone already runs on one core ($1.67\text{--}2.47\times$ across CPUs, no threads), while parallelization adds scaling for many-core runs (up to $5.33\times$ at 24 threads, $7.14\times$ over the stock single-core engine). Correctness is enforced by thread-count invariance—identical output across thread counts—used as an automatable data-race detector. At one thread the engine is byte-identical to the unmodified serial engine; at higher counts, exercising in-stream water quality exposed a compiler-induced data race in particulate constituents (15–20% on three sediment-associated species), which we resolve by a byte-identical land-parallel, routing-serial mode. Strikingly, the single largest speed gain came not from threads but from removing one redundant array reset on the serial main stem—a serial fix that also raised the parallel ceiling. All code, every optimization as an individually validated commit, and the benchmark harness are released.

Keywords: SWAT+; shared-memory parallelism; OpenMP; river routing; reproducibility; model acceleration.

1 Introduction

The Soil and Water Assessment Tool (SWAT) and its restructured successor SWAT+ [Arnold et al., 1998, Bieger et al., 2017] are among the most widely used process-based watershed models. As input data and computing have improved, the community has pushed SWAT+ toward larger domains and hyper-resolution discretizations—tens of thousands of hydrologic response units (HRUs) and channels in a single basin. At that scale the cost of a *single* simulation becomes the binding constraint. The benchmark model used throughout this paper, a USGS HUC-8 watershed (hydrologic unit 03100101) is a regional, hyper-resolution model generated by the SWATGenX platform; it contains 57,998 HRUs and 8,181 channels (full specification in Section 3.1), and one simulated year takes about three minutes on a single core. Calibration—the dominant computational activity in practice—requires hundreds to

thousands of such evaluations, so per-run cost multiplies directly into days of wall-clock time and substantial cloud expense.

1.1 The gap: engine-level vs ensemble-level parallelism

Most prior work on “parallel SWAT” accelerates the *calibration*, not the *model*. Frameworks distribute independent parameter sets across cores or cluster nodes [Rouholahnejad et al., 2012, Zhang et al., 2013, Yalaw et al., 2013], and more recent efforts containerize the same ensemble pattern for reproducibility [Lin et al., 2024]. This ensemble parallelism is essential and we use it, but it has a hard ceiling: population-based calibration of a model with a few tens of adjustable parameters saturates at a few tens of concurrent evaluations [Rafiei et al., 2022]—beyond that, a larger population does not improve the optimum and can degrade it. Once an individual model is large, the only way to apply more than that handful of cores to *one* model is to parallelize the engine itself.

Engine-internal parallelization of SWAT is comparatively rare and has targeted the embarrassingly parallel land phase: OpenMP over HRUs in the gridded SWATG variant [Zhang et al., 2017] and compiler-assisted OpenMP in SWAT [Ki et al., 2015]. The land phase is the easy half. The hard half—channel routing—is a strictly ordered traversal of the river network: a reach cannot be solved until its upstream contributors are. This data dependency is exactly what makes routing the serial bottleneck, and it is the same structural obstacle faced by every distributed routing model [David et al., 2011, Mizukami et al., 2021, Yamazaki et al., 2011].

1.2 Two non-negotiable constraints

A parallel engine intended for real use must satisfy two constraints that the existing literature largely sidesteps. First, results must not change: a calibrated, published, or regulatory model cannot silently produce different numbers because it ran on more cores. Second, correctness must be demonstrable, not asserted—data races in shared-memory scientific codes

are notoriously intermittent and escape casual testing [Atzeni et al., 2016]. We treat both as first-class: the one-thread path reproduces the unmodified serial engine *bit for bit*, and we validate the parallel paths by *thread-count invariance*, a per-output bitwise consistency criterion in the spirit of ensemble consistency testing for Earth-system models [Baker et al., 2015], which here doubles as a precise race detector.

1.3 Contributions

This paper makes the following contributions:

1. A reproducible acceleration methodology for a large process-based Fortran model: `profile` $\rightarrow$ `classify` $\rightarrow$ `fix-by-class` $\rightarrow$ `validate` $\rightarrow$ `re-measure`, with each optimization an independently validated commit.
2. A bottleneck taxonomy of SWAT+ at scale, with the central empirical finding that the dominant cost is *overhead, not hydrology*—per-object array zeroing, struct copies, $\mathcal{O}(N^2)$ name lookups, and a serial routing critical path.
3. A routing-DAG-aware shared-memory parallelization: a topological level wavefront over the engine command order, with reentrancy achieved by privatizing current-object state (`threadprivate`), and a discussion of the Fortran `SAVE`-locals hazard that makes naïve privatization unsafe.
4. A correctness framework: byte-identity at one thread plus thread-count invariance as an automatable race detector, and an honest accounting of a compiler-induced in-stream particulate race (15–20% on three sediment-associated species) and its resolution by a byte-identical mode.
5. An architectural-limit result: a measured characterization of where the SWAT+ daily step is parallelizable (the wide land phase) and where it is bound by the converging main-stem critical path, validated across multiple CPU microarchitectures.

The remainder of the paper reviews related work (Section 2), details the methodology and the parallelization (Section 3), reports scaling and correctness results (Section 4), and discusses the architectural limits and portability (Sections 5–6).

2 Related work

We organize prior work along the axis that matters for this paper: *where* the parallelism is extracted. Three layers are distinguishable—ensemble (many independent runs), spatial-domain decomposition (split the watershed), and engine-internal (parallelize one run’s daily step)—and the serial routing dependency reappears as the central obstacle at every layer.

2.1 Ensemble-level parallelism for calibration

The most mature line of work parallelizes the *calibration loop* rather than the model. Rouhollahnejad et al. [2012] introduced a parallelization framework for SWAT calibration (the basis of the widely used SWAT-CUP workflow), distributing independent parameter sets across cores. Zhang et al. [2013] coupled SWAT to a Python parallel layer (mpi4py/OpenMPI) driving a multi-objective evolutionary optimizer, reporting ensemble speedups of 45–109× on a cluster depending on model complexity. Yalew et al. [2013] parallelized a large spatially distributed model via MPI, and Lin et al. [2024] recently containerized the calibration ensemble for computational reproducibility. These approaches are complementary to ours and remain the right tool for exploring parameter space. Their limitation is structural: population-based search over a model with $\sim$ tens of parameters saturates at $\sim$ tens of concurrent evaluations, so ensemble parallelism cannot reduce the wall-clock cost of an *individual* large simulation, nor apply more than that handful of cores to a single model. That is precisely the regime this paper addresses.

2.2 Engine-internal parallelism in SWAT/SWAT+

Fewer studies parallelize the SWAT engine itself, and those that do concentrate on the land phase, which is embarrassingly parallel across HRUs. Ki et al. [2015] applied Intel-compiler OpenMP to SWAT and benchmarked the result; Zhang et al. [2017] added OpenMP over HRUs to the gridded SWATG variant (“SWATGP”), reporting up to $\sim 9\times$ on a $\sim 2000\text{ km}^2$ watershed at 500 m with 15 threads. Two observations motivate our work. First, these efforts parallelize the land phase but do not tackle the channel-routing critical path, which our profiling (Section 4) shows is roughly $7\times$ the land-phase cost on a large basin—so land-phase-only speedup is Amdahl-bounded by the serial routing remainder. Second, the SWAT+ restructuring [Bieger et al., 2017] replaced subbasins with an object/command architecture, which changes the parallelization problem from “loop over HRUs” to “traverse a heterogeneous object DAG”; to our knowledge a routing-DAG-aware shared-memory parallelization of SWAT+ that preserves bit-level results has not been reported.

2.3 The serial routing dependency in other hydrological models

The obstacle we confront—downstream reaches depend on upstream outflow—is fundamental to river routing and has shaped the design of every continental routing model. The strategies divide cleanly:

- **Matrix linearization (solve all reaches at once).** RAPID [David et al., 2011] casts Muskingum routing on a vector network into a single sparse linear system per time step and solves it with a parallel linear-algebra backend (PETSc), turning the topological dependency into matrix structure. This scales but requires a linear routing scheme and a global solve.
- **Spatial-domain decomposition.** mizuRoute [Mizukami et al., 2021] partitions the network into sub-basins with a two-level hierarchical decomposition for distributed-memory (MPI) execution, and—in its impulse-response mode—parallelizes primarily

in the *time* dimension while preserving sequential topological order. CaMa-Flood [Yamazaki et al., 2011] partitions the global domain into subdomains and advances local inertial equations, updating cells in parallel across space. A recent study parallelizes channel routing by straightforward domain decomposition of the network [Liu et al., 2023].

These are predominantly *distributed-memory* designs that either change the numerics (linearization) or partition the domain across ranks (decomposition), and they generally do not guarantee bitwise identity to a serial reference. Our setting differs in three ways: we keep SWAT+’s existing (nonlinear, sub-stepped) routing numerics unchanged; we target *shared-memory* many-core nodes (the common cloud-calibration unit) rather than clusters; and we make preservation of the serial result a hard requirement. Our mechanism—a topological *level wavefront* that runs all DAG nodes at the same depth concurrently and synchronizes between levels—is the shared-memory analogue of the dependency-respecting ordering those models implement across ranks.

2.4 Correctness and reproducibility of parallel scientific code

Shared-memory parallelization of a large mutable-state Fortran model is dominated by the risk of data races, which are intermittent and poorly exposed by ordinary testing [Atzeni et al., 2016]. Dynamic and static race detectors exist but scale awkwardly to a model of SWAT+’s size and global-state idioms. We instead adopt an output-level criterion: *thread-count invariance*, requiring repeated runs at different thread counts to produce identical daily output. This is conceptually aligned with the ensemble-consistency testing used to certify Earth-system-model ports across compilers and machines [Baker et al., 2015], but applied at bit granularity and used *diagnostically*—a run-to-run difference localizes the exact unprotected state. We also separate genuine races from benign floating-point non-associativity introduced by aggressive compiler optimization (`-ipo`); the latter is a known source of bit-level variation [Intel Corporation, 2026] and is quantified rather than conflated with incor-

167 rectness.

168 3 Methods

169 3.1 Benchmark model and toolchain

170 All measurements use a single benchmark model so that speedups are comparable across
171 optimizations: a single USGS HUC-8 watershed (hydrologic unit 03100101; the Peace River,
172 southwest Florida), a *regional, hyper-resolution* SWAT+ model generated by the SWATGenX
173 platform. The basin is delineated from the USGS National Hydrography Dataset Plus High
174 Resolution (NHDPlus HR) over a 1/3 arc-second (≈ 10 m) DEM, with land cover from the
175 500 m National Land Cover Database (NLCD), soils from gridded SSURGO (gSSURGO),
176 and climate forcing from PRISM (precipitation and temperature) and the National Solar Ra-
177 diation Database (NSRDB). The model comprises 57,998 hydrologic response units (HRUs),
178 8,181 channels, 9,341 routing units (subbasins) resolved into 9,342 landscape units, 326 reser-
179 voirs and lakes, and 170 aquifers. The high HRU count follows from the large drainage area
180 combined with a hyper-resolution slope/land-cover/soil intersection—this is a single regional
181 model, not a continental one, though the SWATGenX platform provides model-building cov-
182 erage across the conterminous United States (CONUS). Unless stated otherwise, the timed
183 workload is one simulated year, which exercises the full daily land-and-routing step over
184 the entire network. The engine is built with the Intel oneAPI Fortran compiler (`ifx`) at
185 `-O3 -ipo`; `gfortran` fails to produce a working binary at this model size, so `ifx` is the
186 reference toolchain. OpenMP is enabled with `-fopenmp`. We keep two binaries—the un-
187 modified serial engine and the parallel engine—so the parallel work can be checked against
188 an exact reference; at one thread the parallel engine executes the original serial code path
189 (Section 3.6).

190 3.2 The acceleration loop

191 The methodology is a disciplined iteration, applied until returns diminish:

- 192 1. **Profile.** Intel VTune **hotspots** collection on a representative window (180 simulated
193 days), reported per routine and per source line, to find where time is actually spent
194 rather than where it is assumed to be spent.
- 195 2. **Classify.** Assign each hotspot to a class—startup name-matching, per-object re-
196 initialization, derived-type copies, serial critical path, or genuine hydrologic compute—
197 because the fix differs by class and the classification is the transferable result.
- 198 3. **Fix by class.** Apply the class-appropriate change (e.g. index a name lookup, scope
199 an array reset, privatize shared scratch, restructure for parallelism).
- 200 4. **Validate.** Re-run the correctness battery (Section 3.6) before accepting any change.
- 201 5. **Re-measure.** Re-time and re-profile; the new dominant hotspot becomes the next
202 target. Each accepted change is a single version-control commit whose message records
203 its validation evidence.

204 3.3 The routing DAG and the level wavefront

205 SWAT+ advances one daily time step by walking every object—HRU, routing unit, channel,
206 reservoir, aquifer—in a single fixed sequence (the *command order*). That sequence encodes
207 the river network’s connectivity: an object appears after all objects that feed it. Equivalently,
208 the command order is a topological sort of a directed acyclic graph (DAG) whose edges are
209 the hydrologic routing connections.

210 A topological sort admits a *level* (antichain) decomposition: level ℓ contains every object
211 all of whose upstream dependencies lie in levels $< \ell$. Objects within a level are mutually
212 independent and may be computed concurrently; levels must be processed in order. We
213 precompute this level index once per run from the command order and drive the daily step

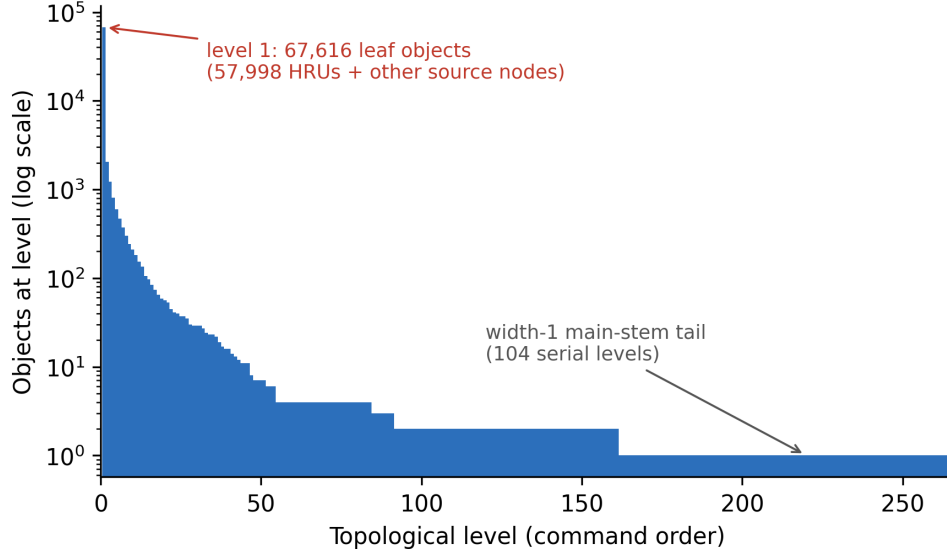


Figure 1: The command-order level decomposition for the benchmark river network (265 levels). Level 1 contains all 67,616 objects with no upstream dependency—the 57,998 HRUs together with the other source (leaf) nodes (point-source/recall objects and headwater units)—an enormously wide, abundantly parallel level; the levels narrow monotonically as the channel network converges, reaching width 1 at the deep main-stem reaches (104 successive width-1 levels). The length of this width-1 tail, not the per-level synchronization, sets the Amdahl ceiling on single-run speedup (Section 5).

as a level wavefront: for each level, an OpenMP parallel loop computes its objects; an implicit barrier between levels preserves the upstream-before-downstream ordering. For the benchmark network this yields 265 levels: level 1 is the entire land phase ($\approx 58,000$ HRUs—an enormous wide level with abundant parallelism), and the levels narrow as the channel network converges, reaching width 1 at the deep main-stem reaches—the serial critical path that ultimately bounds the achievable speedup (Figure 1; Section 5).

Two engineering refinements proved necessary. First, forking a parallel region *per level* (265 times per day) made thread-management overhead dominate and caused an *8-thread regression*; we instead open one parallel region per simulated day and drive the levels inside it, with the per-level barrier as the only synchronization. Second, the long tail of width-1 (and other very narrow) levels paid one barrier each for almost no parallel work; we fuse consecutive narrow levels into a single master-thread serial block, so the deep main stem

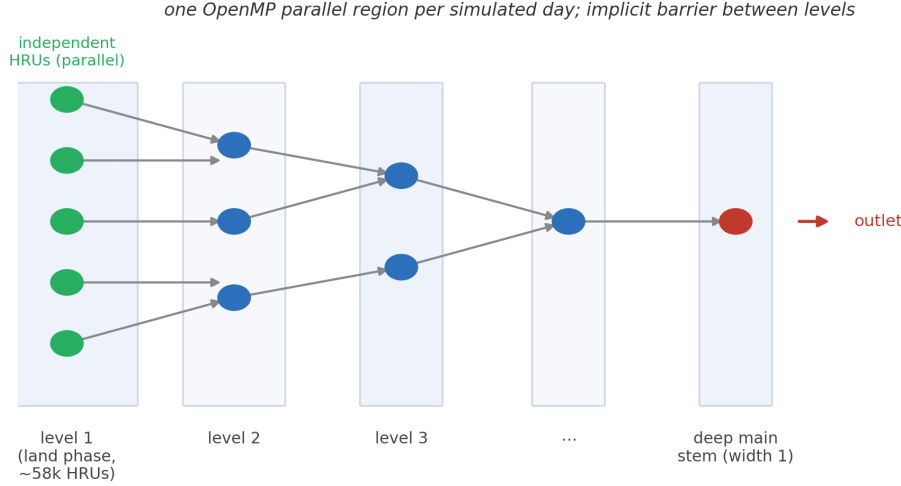


Figure 2: The routing-DAG level wavefront. The daily command order is a topological sort of the routing DAG; objects at the same level are independent and run concurrently under a single per-day OpenMP parallel region, with an implicit barrier between levels enforcing upstream-before-downstream order. Consecutive narrow (near-width-1) levels on the main stem are fused into one serial block to avoid paying a barrier per level. At one thread the wavefront is inactive and the original serial command order executes unchanged.

226 pays one barrier instead of one-per-level (Figure 2).

227 3.4 Reentrancy: privatizing current-object state

228 The obstacle to running objects concurrently is not the routing math—it is that the engine,
 229 though written in modern Fortran, communicates through *module-global* “*current-object*”
 230 *scratch* by design: a routine sets a global index (“the HRU we are on”, “the channel we
 231 are on”) and downstream routines read it. Under threads, concurrent objects clobber that
 232 shared state between the set and the use. Making the per-object body reentrant therefore
 233 required auditing every such variable and giving each thread its own copy via OpenMP
 234 **threadprivate**. Examples that had to be privatized include the current weather-station
 235 index (a shared read of `iwst` between its set and use caused $\sim 0.2\%$ PET drift on a station-
 236 dependent subset of HRUs), the channel dispatch indices and rating-curve/peak-rate scratch,
 237 and a large cluster of plant/residue “temporary storage” arrays.

238 A Fortran-specific hazard complicates this: a local declared with an initializer is *implied-*

itly **SAVED**, hence shared across calls and therefore across threads, so the reflexive “strip the initializer” fix is correct only for write-before-read scratch; the few accumulators that legitimately persist must be kept **SAVED** *and* marked **threadprivate**. This subtlety, and the one-thread byte-identity check that caught it, are detailed in Supplementary Section S2.

3.5 Targeted hotspot removal (overhead, not hydrology)

Profiling repeatedly surfaced *non-hydrologic* hotspots, each removed without touching the science: an $\mathcal{O}(N^2)$ string name-match at startup (replaced by an $\mathcal{O}(1)$ index), a per-object whole-array re-zeroing in the stream-temperature routine (**ch_temp**, which re-zeroed the entire landscape-unit array once per channel per day—the single hottest line in the engine), and redundant derived-type struct copies in inflow-hydrograph handling. Each was scoped or indexed to the data actually used, leaving output bit-identical; Section 4.1 reports the per-rung effect.

3.6 Correctness framework

We validate with two complementary criteria.

Byte-identity at one thread. At one thread the wavefront is inactive and the engine executes the original command order, so its daily output must match the unmodified serial engine *bit for bit*. We verify this with exact byte-level diffs of every daily HRU and channel record on a one-year benchmark run. This check caught a small ($\sim 0.3\%$) *deterministic* residual introduced by the reentrancy refactor in two routines (**ch_watqual4**, **et_pot**), where stripping **SAVE** had dropped persistent scratch values; restoring them as **threadprivate** **SAVE** state made one-thread output bit-identical again (Section 3.4).

Thread-count invariance as a race detector. For $N > 1$ we require repeated runs at different thread counts (1-vs-4, 4-vs-4, 4-vs-8) to produce identical daily output. Because

a correct shared-memory program is deterministic regardless of how work is scheduled, any run-to-run difference is, by definition, a real data race—and the specific output fields that differ point directly at the unprotected state. We use a strict tripwire (any record off by $> 10^{-6}$ counts as a difference). This criterion is what drove the iterative race-elimination in Section 3.4 and is fully automatable.

Separating compiler noise from incorrectness. The `-ipo` production build reorders floating-point operations, so two algorithmically identical builds can differ at the last bits. We therefore distinguish (i) a non-`ipo` build, which is bit-identical to the serial source and proves the *algorithm* is unchanged, from (ii) the `-ipo` build, whose residual ($\sim 5 \times 10^{-4}$ relative, $\approx 0.008\%$) is compiler floating-point reassociation, not a logic change. Exercising the full process chain exposed one $N > 1$ data race in the in-stream *particulate* constituents (Section 4.4), localized to the channel constituent-transport path and under active resolution; hydrology and dissolved species are model-equivalent across thread counts. The stream-temperature column is a separate pre-existing upstream out-of-bounds read, output-only in the non-gwflow path, preserved deliberately to match the reference serial behavior.

4 Results

We report results as a *cumulative optimization ladder*: starting from the unmodified stock engine, each optimization *class* is added in a fixed order and its marginal contribution measured on the same model, box, and protocol (Section 3, Table 1). This isolates two qualitatively different products: Group A optimizations remove serial overhead and so benefit *every* user on a single core with no behavior change; Group B (parallelization) benefits users who apply many cores to a single model. All rungs—including the I/O rungs whose techniques were introduced in prior work [the NetCDF backend and channel print filter, benchmarked there; Rafiei, 2026]—are re-measured here on the 57,998-HRU benchmark, because their prior figures were obtained on a different (94k-HRU) model and do not transfer;

Table 1: Cumulative optimization ladder on the HUC-8 benchmark model (57,998 HRUs, one simulated year, `ifx -O3 -ipo`, dedicated c8a.8xlarge, best of two). Each rung adds one class; “marginal” is the speedup from the preceding rung, “cumulative” is versus stock. All $N=1$ except the final-engine parallel row (Group B).

Rung		Class	Wall $N=1$ (s)	Marginal	Cumulative
R0	Stock engine	baseline	191.9	$1.00\times$	$1.00\times$
	+ hru_read O(1) index	serial: startup	185.4	$1.03\times$	$1.03\times$
	+ varinit per-row reset	serial: per-step	171.0	$1.08\times$	$1.12\times$
	+ drop inflow struct copy	serial: channel	137.6	$1.24\times$	$1.39\times$
	+ ch_temp reset scope	serial: channel	138.9	$0.99\times$	$1.38\times$
R6	+ final (narrow-fuse, $N=1$)	serial: final	115.2	$1.21\times$	$1.67\times$
<i>Group B — parallelization of the final engine (dedicated AWS instances, Table 2):</i>					
R6	final engine, parallel	parallel	—	$5.33\times$	$\approx 7.14\times$

Wall times: dedicated c8a instance, one simulated year, daily `channel.sd` output, best of 2. I/O (NetCDF backend, an extension from our fork [Rafiei, 2026], re-measured here): stock text output 222.8 s $\rightarrow$ NetCDF 196.8 s (1.13×). The Group B factor is the final engine’s peak self-relative thread speedup (5.33× at 24 threads); the combined stock-to-parallel $\approx 7.14\times$ is measured end-to-end against the stock engine on the same machine (Table 2, §4.5).

this keeps the stock $\rightarrow$ final comparison internally consistent.

4.1 Group A: serial overhead removal (one core, every user)

The Group A rungs change *no* model behavior and require *no* threads; they make the engine everyone already runs faster. The central empirical finding is that these gains are large because the stock engine spends much of its time on *overhead*, *not hydrology*:

- **I/O.** Re-encoding output from text to a NetCDF backend (a fork extension benchmarked in the companion paper [Rafiei, 2026], re-measured here) cuts disk traffic: stock text output 222.8 s $\rightarrow$ NetCDF 196.8 s, 1.13× at $N=1$.
- **Startup name-matching (`hru_read`).** HRU input was matched by an $O(N)$ linear name search, $O(N^2)$ over the dataset; an $O(1)$ index removes it, output-identical (1.03×, 191.9 $\rightarrow$ 185.4 s).
- **Per-step re-initialization (`varinit`).** A daily $O(N_{\text{hru}}^2)$ whole-array `memset` in variable initialization was scoped to the current HRU’s row (1.08×, 185.4 $\rightarrow$ 171.0 s).

- **Channel per-step overhead.** Removing a redundant inflow-hydrograph struct copy was the largest single serial step ($1.24\times$, $171.0\rightarrow 137.6$ s). The `ch_temp` reset—the single hottest line in the engine, re-zeroing the *whole* landscape-unit array once per channel per day—was scoped to the units each channel uses; notably this barely moves the *serial* time ($0.99\times$, $137.6\rightarrow 138.9$ s) because its real effect is on the *parallel* critical path (Section 4.2: removing it from the serial main stem markedly raised the multi-thread ceiling by shrinking the Amdahl serial fraction). This split—a fix that is nearly free serially but decisive in parallel—is itself a finding.

Group A headline ($N=1$, no threads): a cumulative $1.67\times$ over the stock engine ($191.9\rightarrow 115.2$ s) on the c8a reference node, a free acceleration for the entire existing user base independent of whether they ever enable threads. Because these gains attack memory-bound overhead, they are *larger* on the more memory-starved parts—up to $2.47\times$ on the Intel “Granite Rapids” node (Table 2)—so $1.67\times$ is a conservative floor.

4.2 Group B: parallelization (scaling a single model)

On top of the serial-optimized engine, the routing-DAG level wavefront (Section 3.3) adds thread parallelism. Because Group A removed the dominant serial channel hotspot, the parallel speedup is no longer suppressed by it: removing the `ch_temp` reset on the critical path shrinks the Amdahl serial fraction and so lifts the achievable scaling, and narrow-level fusion (one barrier for the deep main stem instead of one-per-level) lifts it further. This interaction—a serial fix raising the *parallel* ceiling—is itself a result (Section 5). All scaling is measured on quiet, dedicated AWS instances (one model per box, thread-pinned, best of two); we report no shared- or contended-host timings.

The scaling is bounded by *physical cores and memory bandwidth*, not thread count: SWAT+ at this scale is memory-bound, so once enough cores saturate the wide land-phase level, additional cores only wait at the narrow routing levels, and parts regress onto SMT siblings past their physical-core count (Figure 3; Section 5).

Table 2: Cross-hardware scaling of the final engine (each machine’s own $1 \rightarrow N$ ratio), identical engine, one-simulated-year benchmark workload, daily `channel_sd` output, thread-pinned, best of two, on dedicated AWS instances. t_1 = one-thread wall on the parallel binary. Boldface marks each CPU’s peak; beyond it, threads land on SMT siblings and contend.

	c8a (AMD EPYC “Turin”)	c8i (Intel Xeon “Granite Rapids”)	c5a (AMD EPYC “Rome”)
physical cores (vCPU)	32 (32)	16 (32)	16 (32)
SMT	off (1 thr/core)	on (2 thr/core)	on (2 thr/core)
stock R_0 (s, $N=1$)	191.9	272.2	350.6
serial gain ($R_0 \rightarrow R_6$)	$1.67\times$	$2.47\times$	$2.16\times$
t_1 (s, parallel binary)	143.1	130.1	202.5
2 threads	$1.70\times$	$1.59\times$	$1.62\times$
4 threads	$2.79\times$	$2.41\times$	$2.47\times$
8 threads	$3.95\times$	$3.29\times$	$3.21\times$
12 threads	$4.54\times$	$3.75\times$	$3.60\times$
16 threads	$4.90\times$	$4.01\times$	$3.81\times$
20 threads	$5.18\times$	$3.73\times$	$3.51\times$
24 threads	$5.33\times$	$3.74\times$	$3.58\times$

4.3 The bottleneck migrates across the campaign (VTune)

Profiling at three points makes the campaign legible (Figure 4). In the *stock* engine, channel routing and the to-be-removed overhead dominate; after Group A, the `ch_temp` hotspot collapses (its hottest line falls from ≈ 731 s to ≈ 0.8 s) and no channel routine is dominant; in the *parallel* final engine at eight threads, the bottleneck has migrated from computation to *synchronization*—of the $\approx 1,267$ CPU-seconds the threads consumed, only $\approx 73\%$ is effective work, $\approx 25\%$ is spin idling at level barriers while the serial main stem is walked, and $\approx 2\%$ is overhead. The effective work that remains is dominated by serial channel routing, which costs roughly $7\times$ the parallelizable land phase; the land phase is only $\approx 12\%$ of compute, so the channel main stem—not more threads—is the remaining lever.

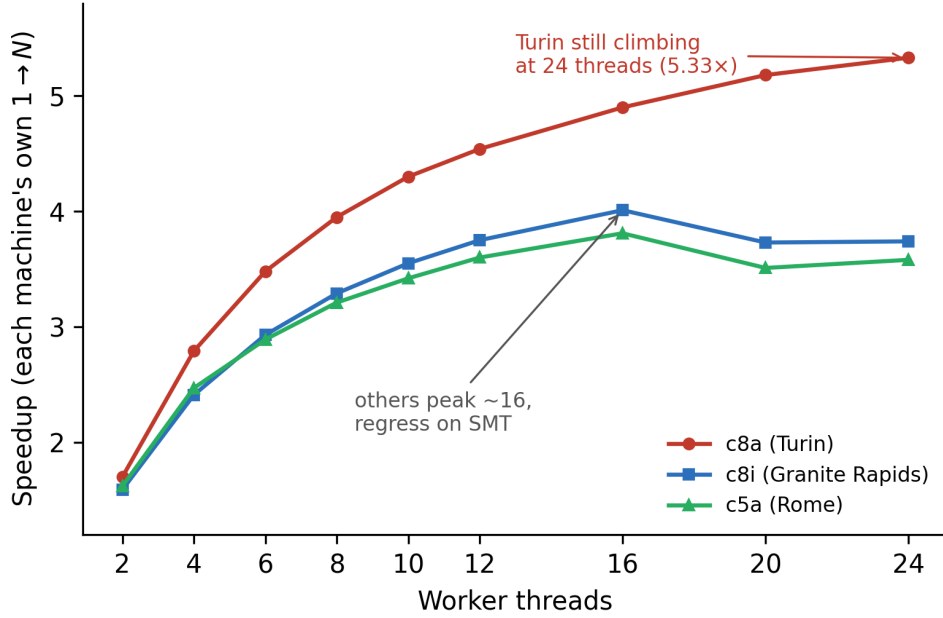


Figure 3: Cross-hardware scaling of the final engine on dedicated AWS instances. The workload is memory-bound: speedup tracks physical cores and memory bandwidth, and parts regress once they run onto SMT siblings.

4.4 Correctness: bit-identity at one thread; a localized particulate race at higher counts

At one thread the parallel engine is byte-identical to the unmodified serial engine (verified bit-for-bit on a one-year run), so an exact reference run is always available. At higher thread counts, *hydrology and dissolved constituents are model-equivalent*: flow, sediment, nitrate, and soluble-phosphorus basin loads agree to $<2\%$ (mostly $<0.5\%$). The open item is the in-stream *particulate* constituents.

Reaching even this required treating *thread-count invariance as a diagnostic* rather than a checkbox (Section 3.6) and exercising the full constituent chemistry: a streamflow-only test passes while a latent race in the particulate constituents goes undetected. Only with in-stream water quality active over a multi-year run does thread-count invariance expose a 15–20% divergence in three *particulate/sediment-associated* species (organic-N, ammonia, sediment-P), with run-to-run variation confirming a data race rather than reassociation.

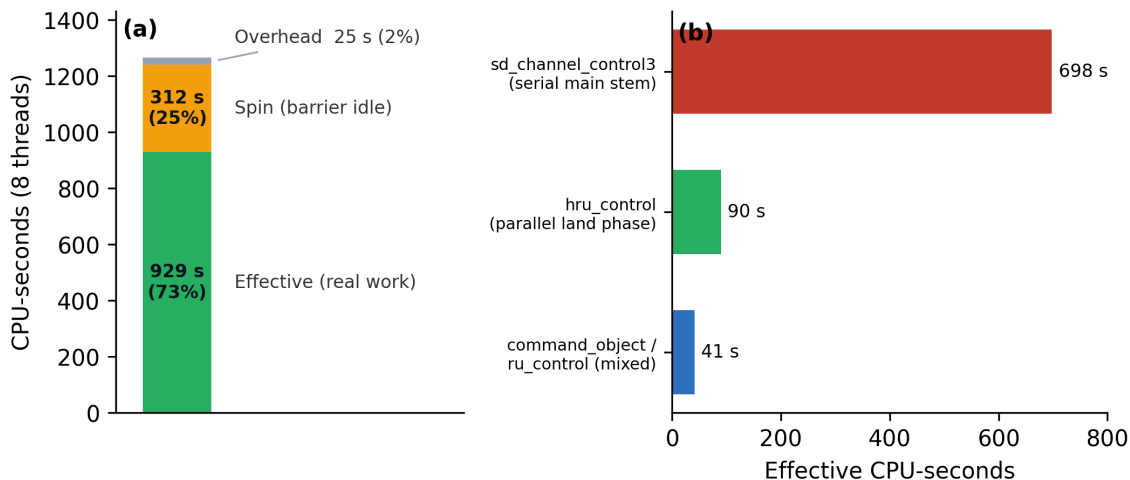


Figure 4: Intel VTune profile of the final engine at eight threads (180-day window). (a) CPU-time composition: only $\approx 73\%$ of the $\approx 1,266$ thread-seconds is effective work; $\approx 25\%$ is barrier spin idling against the serial main stem and $\approx 2\%$ is scheduling/`threadprivate` overhead—the limit has migrated from computation to synchronization. (b) Effective work by routine: serial channel routing (`sd_channel_control3`, 698 s) dominates the parallelizable land phase (`hru_control`, 90 s) by roughly $7.7\times$, so the main stem—not more threads—is the remaining lever. (The Group A serial pass earlier collapsed the dominant `ch_temp` hotspot from ≈ 731 s to ≈ 0.8 s; that hotspot is absent here.)

The HRU land phase is byte-identical across thread counts and the dissolved species (NO_3 , soluble-P) are spared, localizing the race to the channel in-stream constituent-transport path.

Root cause (compiler, not algorithm). ThreadSanitizer traces it not to a missing `threadprivate`—every per-object buffer is privatized—but to `ifx` allocating the *temporaries* of the 152-byte `hyd_output` derived type (operator results, function returns, argument copy-ins) in shared static module storage rather than on the per-thread stack, so concurrent channels clobber the same static temporary. The behavior is invariant to every compiler control we tried, and a field-by-field source rewrite is byte-identical at one thread but does not remove it (array-element arguments re-create static copy-in temporaries). The full forensic trail is in Supplementary Section S3.

Resolution: a byte-identical mode at a measured cost. Because the race lives entirely in the routing phase, running the land phase in parallel and the routing phase seri-

ally (`SWATPLUS_ROUTING_SERIAL=1`) is race-free and byte-identical—thread-count-invariant to the floating-point-reassociation floor. The cost is the routing phase’s parallelism: the byte-identical mode scales $2.04\times$ at 4 threads / $2.50\times$ at 8 versus the full wavefront’s $2.79\times$ / $3.98\times$, a $\approx 37\%$ forfeit at eight threads that grows with thread count (Supplementary Table S3). The engine thus offers two modes: byte-identical for in-stream water quality, and the full wavefront (bit-exact for flow/sediment/dissolved species) otherwise. A non-ipo build is bit-identical to the serial source; the -ipo residual ($\sim 0.008\%$) is reassociation, not the race.

4.5 Putting it together: a speedup configurator

Every speedup in this paper is measured against one fixed reference—the stock engine, one thread, writing daily `channel_sd` for all reaches—and the optimized and parallel runs write the *identical* output; we deliberately do not compare a fast filtered run against a slow full-export run, which would fold an output-configuration choice into the engine number. The total acceleration a practitioner sees is the product of three *independent* layers, and conflating them is the most common way these numbers are misread: (i) output scope, an I/O effect *outside* the engine—full daily export of every object is several-fold slower than the filtered calibration profile (of order $3\text{--}6\times$, growing with model size; companion study [Rafiei, 2026]) and is excluded from every figure here; (ii) the machine-portable serial gain ($1.67\text{--}2.47\times$, Group A); and (iii) the thread scaling above (Group B). Because the fixed-output baseline is already compute-bound, (ii) and (iii) compose on top of (i). Figure 5 shows the directly-measured stock→parallel total on the three CPUs plus the byte-identical mode; an interactive version is published with the open engine, and the full layer composition is tabulated in Supplementary Section S4.

4.6 Relevance to calibration

Single-run acceleration is the lever that population-based calibration cannot otherwise reach. Ensemble parallelism—running many independent evaluations concurrently—saturates at a

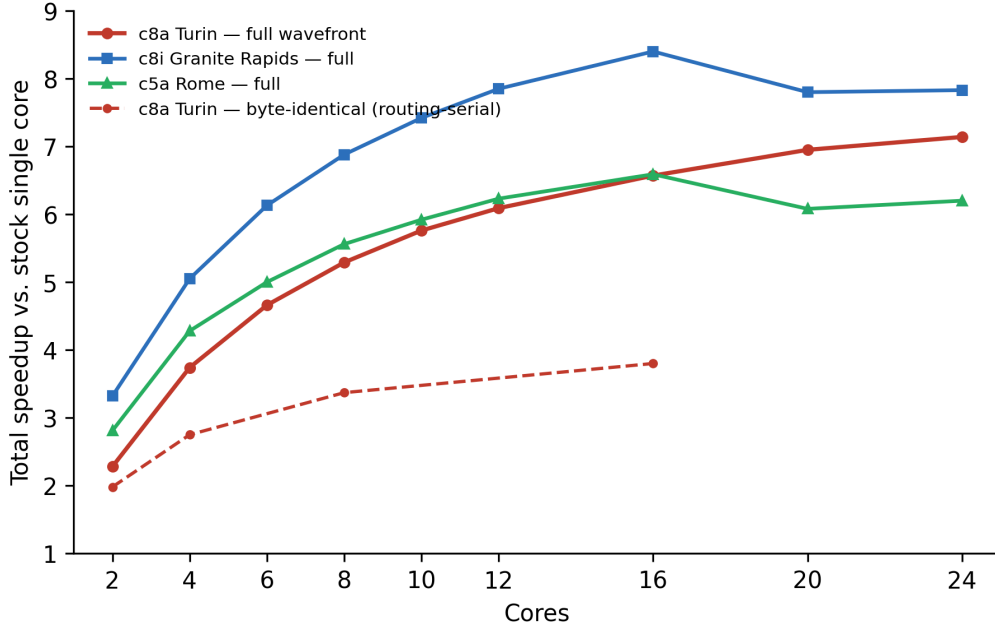


Figure 5: End-to-end engine speedup versus the stock single-core engine as a function of cores, directly measured on three dedicated AWS CPUs (full wavefront), plus the byte-identical routing-serial mode on the c8a node (dashed). The $1\times$ reference is the *stock engine at one thread writing daily `channel_sd` for all reaches*; every curve writes that same output, so the speedup is pure engine work, not reduced printing. Output-scope savings (layer i; gauge filtering, NetCDF) multiply on top and are deliberately excluded here.

few tens of concurrent members [Rafiei et al., 2022], beyond which additional cores cannot help a fixed population; accelerating the *individual* simulation is then the only way to put more cores on one model. The two contributions land differently here: the Group A serial gains accelerate every single evaluation with no threads and no behavior change, while the parallel engine lets a calibration apply cores past the ensemble ceiling to each member.

5 Discussion

5.1 Two products, cleanly separated

The cumulative ladder (Table 1) deliberately separates two contributions that a single blended speedup would conflate. Group A—I/O reduction and serial overhead removal—changes no model behavior and needs no threads, so it accelerates the engine *everyone already*

runs and ships to all users through the default binary. Group B —the parallelization— benefits only those who put many cores on a single model. The distinction matters for adoption: a practitioner on a laptop gains from Group A for free, while a practitioner calibrating a large basin on a many-core node additionally gains from Group B. The single most important practical message of this paper is that a large fraction of the acceleration of the stock engine is achievable *without parallelism at all*.

A cumulative ladder carries one caveat: a rung’s marginal contribution depends on the order rungs are added. We fix a logical order and report marginals *given that order*; the cumulative stock→final factor is order-independent and is the headline number.

5.2 A serial fix can raise the parallel ceiling

The ladder also exposes an interaction that single-axis studies miss. The `ch_temp` reset is a Group A (serial) fix, yet its largest visible effect was on *parallel* scaling: because it sat on the serial main-stem critical path, removing it shrank the serial fraction and substantially lifted the multi-thread speedup ceiling. The lesson is that on an irregular DAG with a long serial tail, serial-overhead removal and parallelization are not independent—reducing critical-path work is often the most effective way to improve parallel efficiency, more so than adding threads. This is why we measure each serial fix at $N = 1$ (its pure serial value) *and* report its effect on the scaling curve.

5.3 Where SWAT+ is parallelizable, and where it is not

The scaling ceiling is not an implementation defect; it is the river network. The daily step decomposes into a very wide land-phase level ($\approx 58,000$ independent HRUs) followed by a routing cascade that narrows monotonically to a width-1 main stem. By Amdahl’s law [Amdahl, 1967] the achievable speedup is bounded by the serial fraction, and after the `ch_temp` fix that fraction is dominated by the *length* of the converging main-stem path—the 104 successive width-1 levels of Figure 1—rather than by per-level synchronization. This

is visible in the cross-hardware data: machines stop scaling at their physical-core count because, once enough cores exist to saturate the wide levels, additional cores only wait at the narrow ones. The practical implication is that for SWAT+, single-run acceleration past the land-phase regime requires either (i) overlapping independent sub-trees of the routing DAG more aggressively across the narrow levels, or (ii) reducing the per-channel arithmetic on the main stem (the in-stream water-quality and sediment routines), not simply adding threads.

5.4 Memory bandwidth, clock, and the meaning of “speedup”

SWAT+ at scale is memory-bound: the cross-hardware results show real cores beating cores-plus-SMT and 32-core Turin out-scaling 16-core parts. Two reporting hazards follow. First, hardware that turbo-boosts a single thread and down-clocks under load deflates the $1 \rightarrow N$ ratio, so the reported speedup understates the delivered parallel work. Second, CPU-utilization monitors mislead—25% of busy time at eight threads was barrier spin, not work. We therefore report wall time and best-of- N alongside ratios, and we recommend the same discipline for comparable studies.

5.5 Correctness as a methodology

Treating thread-count invariance as the primary acceptance test, rather than spot-checking a hydrograph, converted an open-ended race hunt into a localized, repeatable procedure in which each failing output field named the unprotected state. It exposed two defects ordinary checks miss: an implicit-SAVE Fortran initializer, and—only once in-stream water quality was exercised over a multi-year run—a compiler-induced race in the particulate constituents (Section 4.4). The transferable lesson is twofold: an equivalence test must exercise the full process chain or it certifies only the subset it touches; and when a compiler will not stack-allocate large derived-type temporaries, the safe granularity of shared-memory parallelism is bounded by where those temporaries are shared.

5.6 Portability and reproducibility

The optimizations are standard OpenMP and portable Fortran; no machine-specific intrinsics are used. The dependence on `ifx` reflects only that `gfortran` fails to build the benchmark model at this size, not a hard requirement of the method. Every optimization is a single commit carrying its own validation evidence, and the benchmark harness regenerates the scaling tables, so the result chain is auditable end to end.

5.7 Limitations

The benchmark is a single (large) basin; while the bottleneck taxonomy should generalize to any SWAT+ model with a deep routing network, the exact level histogram and the land-vs-channel split are basin-specific, and small basins with shallow networks will see different ceilings. The parallel path is validated for the non-gwflow configuration; the coupled groundwater (`gwflow`) module is currently serial and disables the full-DAG wavefront, so groundwater-surface-water runs do not yet benefit (a clearly scoped extension). Finally, the cross-hardware study covers three microarchitectures but a single model size; a size-vs-scaling sweep would strengthen the generality claim.

6 Conclusions

We accelerated the SWAT+ engine on a regional, hyper-resolution HUC-8 benchmark (57,998 HRUs / 8,181 channels) without changing model results, reaching a measured $5.33\times$ speedup at 24 threads on a dedicated AMD “Turin” cloud node (about 3 min $\rightarrow$ $\sim$ half a minute per simulated year) and a combined $\approx 7.14\times$ over the stock single-core engine. The central empirical result is that the dominant cost at scale is *overhead*, *not hydrology*: the single largest improvement came from removing one redundant per-channel array reset on the serial critical path, not from adding cores. The parallelization is a routing-DAG-aware shared-memory level wavefront over the engine command order, made safe by privatizing

current-object state, and its correctness is established by byte-identity to the serial engine at one thread and by thread-count invariance—an automatable race detector—at higher thread counts.

Methodologically, the contribution is a transferable loop for accelerating large process-based scientific codes (profile $\rightarrow$ classify $\rightarrow$ fix-by-class $\rightarrow$ validate $\rightarrow$ re-measure) together with a bottleneck taxonomy and a correctness framework that distinguishes genuine races from benign compiler floating-point reassociation. Scientifically, the contribution is a precise characterization of where SWAT+’s daily step is parallelizable (the wide land phase) and where it is architecturally serial (the converging main-stem routing path), validated across multiple CPU microarchitectures. Because ensemble calibration saturates at a few tens of concurrent members [Rafiei et al., 2022], accelerating the individual simulation is the only way to apply more cores to a single model. All code, per-optimization commits, and benchmarks are released for reproduction and extension.

Code and data availability

The SWAT+ engine fork, with every optimization as a separate commit carrying its validation evidence in the commit message, is public at <https://github.com/rafiei-vahid/swatplus>. The benchmark harness, the measured result files, the figure-generation script, and the benchmark model (Peace River HUC-8 03100101, generated by the SWATGenX platform) are released at <https://github.com/SWATGenX/swatplus-engine-acceleration>. The per-optimization commit map and the build recipe are given in Supplementary Section S1.

Acknowledgements

Benchmark timings were obtained on dedicated Amazon Web Services (AWS) EC2 instances funded by the author. The author thanks the SWAT+ developer community for the open

engine on which this work builds.

Author contributions

V.R. designed the methodology, implemented the optimizations and parallelization, ran the benchmarks, and wrote the manuscript.

Competing interests

The author develops the SWATGenX model-generation platform, which deploys the engine described here; no other competing interests are declared.

References

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967. doi: 10.1145/1465482.1465560.

J. G. Arnold, R. Srinivasan, R. S. Muttiah, and J. R. Williams. Large area hydrologic modeling and assessment part i: Model development. *Journal of the American Water Resources Association*, 34(1):73–89, 1998. doi: 10.1111/j.1752-1688.1998.tb05961.x.

Simone Atzeni, Ganesh Gopalakrishnan, Zvonimir Rakamaric, Dong H. Ahn, Ignacio Laguna, Martin Schulz, Gregory L. Lee, Joachim Protze, and Matthias S. Müller. ARCHER: Effectively spotting data races in large OpenMP applications. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 53–62, 2016. doi: 10.1109/IPDPS.2016.68.

Allison H. Baker, Dorit M. Hammerling, Michael N. Levy, Haiying Xu, John M. Dennis, Brian E. Eaton, Jim Edwards, Cecile Hannay, Sheri A. Mickelson, Richard B.

- Neale, Doug Nychka, John Shollenberger, Joe Tribbia, Mariana Vertenstein, and Dave Williamson. A new ensemble-based consistency test for the Community Earth System Model (pyCECT v1.0). *Geoscientific Model Development*, 8(9):2829–2840, 2015. doi: 10.5194/gmd-8-2829-2015.
- Katrin Bieger, J. G. Arnold, Hendrik Rathjens, Michael J. White, D. D. Bosch, P. M. Allen, Martin Volk, and Raghavan Srinivasan. Introduction to SWAT+, a completely restructured version of the soil and water assessment tool. *Journal of the American Water Resources Association*, 53(1):115–130, 2017. doi: 10.1111/1752-1688.12482.
- Cédric H. David, David R. Maidment, Guo-Yue Niu, Zong-Liang Yang, Florence Habets, and Victor Eijkhout. River network routing on the NHDPlus dataset. *Journal of Hydro-meteorology*, 12(5):913–934, 2011. doi: 10.1175/2011JHM1345.1.
- Intel Corporation. Intel oneapi dpc++/c++ and fortran compiler (ifx), 2026. Toolchain reference.
- Seo Jin Ki, Tak Sugimura, and Albert S. Kim. OpenMP-accelerated SWAT simulation using Intel C and FORTRAN compilers: Development and benchmark. *Computers & Geosciences*, 75:66–72, 2015. doi: 10.1016/j.cageo.2014.10.017.
- Qiaoying Lin, Dejian Zhang, Jiefeng Wu, Xingwei Chen, Yihui Fang, and Bingqing Lin. PASS4SWAT: Orchestration of containerized SWAT for facilitating computational reproducibility of model calibration and uncertainty analysis. *Environmental Modelling & Software*, 178:106085, 2024. doi: 10.1016/j.envsoft.2024.106085.
- Yong-He Liu, Zong-Liang Yang, and Pei-Rong Lin. Parallel river channel routing computation based on a straightforward domain decomposition of river networks. *Journal of Hydrology*, 625:129988, 2023. doi: 10.1016/j.jhydrol.2023.129988.
- Naoki Mizukami, Martyn P. Clark, Shervan Gharari, Erik Kluzek, Ming Pan, Peirong Lin, Hylke E. Beck, and Dai Yamazaki. A vector-based river routing model for Earth system

models: Parallelization and global applications. *Journal of Advances in Modeling Earth Systems*, 13(6):e2020MS002434, 2021. doi: 10.1029/2020MS002434.

Vahid Rafiei. Automating high-resolution SWAT+ watershed model generation from NHDPlus HR and national environmental datasets. *Computers & Geosciences*, 2026. Manuscript submitted for publication; *Computers & Geosciences* CAGEO-D-26-00981.

Vahid Rafiei, A. Pouyan Nejadhashemi, Shahbaz Mushtaq, Ryan T. Bailey, and Duc-Anh An-Vo. An improved calibration technique to address high dimensionality and non-linearity in integrated groundwater and surface water models. *Environmental Modelling & Software*, 149:105312, 2022. doi: 10.1016/j.envsoft.2022.105312.

Elham Rouholahnejad, Karim C. Abbaspour, M. Vejdani, R. Srinivasan, R. Schulin, and A. Lehmann. A parallelization framework for calibration of hydrological models. *Environmental Modelling & Software*, 31:28–36, 2012. doi: 10.1016/j.envsoft.2011.12.001.

S. Yalew, A. van Griensven, N. Ray, L. Kokoszkiewicz, and G. D. Betrie. Distributed computation of large scale SWAT models on the Grid. *Environmental Modelling & Software*, 41:223–230, 2013. doi: 10.1016/j.envsoft.2012.08.002.

Dai Yamazaki, Shinjiro Kanae, Hyungjun Kim, and Taikan Oki. A physically based description of floodplain inundation dynamics in a global river routing model. *Water Resources Research*, 47(4):W04501, 2011. doi: 10.1029/2010WR009726.

Xuesong Zhang, Peter Beeson, Robert Link, David Manowitz, R. C. Izaurralde, Ali Sadeghi, Allison M. Thomson, Ritvik Sahajpal, Raghavan Srinivasan, and Jeffrey G. Arnold. Efficient multi-objective calibration of a computationally intensive hydrologic model with parallel computing software in Python. *Environmental Modelling & Software*, 46:208–218, 2013. doi: 10.1016/j.envsoft.2013.03.013.

Ying Zhang, Jingwen Hou, Yongheng Cao, Juan Gu, and Chunlin Huang. OpenMP paral-

564 lization of a gridded SWAT (SWATG). *Computers & Geosciences*, 109:228–237, 2017.
565 doi: 10.1016/j.cageo.2017.08.002.